

# LISTAS EM PYTHON [ ]

Guia completo com exemplos, resultados e explicações

## Prof. Gustavo Franz (Science/Biology)

*Python Developer in Progress*

*Building Educational Solutions with Python*

GitHub: [github.com/GustaFranz](https://github.com/GustaFranz)

## O que são listas?

- ✓ Listas são coleções **ordenadas** e **mutáveis**.
- ✓ Podem armazenar diferentes tipos de dados.
- ✓ Permitem elementos duplicados.
- ✓ Usamos colchetes [ ] para criar listas.
- ✓ Os índices começam em 0.
- ✓ Muito usadas para armazenar e manipular conjuntos de dados.

## Criando listas

### Formas de criar uma lista

```
numeros = [1, 2, 3, 4, 5] # lista de números
frutas = ['maca', 'banana', 'laranja'] # lista de strings
mista = [10, 'Python', 3.14, True] # tipos diferentes
vazia = [] # lista vazia
repeticao = [0] * 5 # [0, 0, 0, 0, 0]
copia = numeros.copy() # cópia da lista
```

## Operações e análise de listas

Exemplos considerando **numeros = [10, 20, 30, 40, 50]**.

| Comando                   | Resultado    | Explicação                                           |
|---------------------------|--------------|------------------------------------------------------|
| <code>numeros[0]</code>   | 10           | O índice 0 corresponde ao primeiro elemento.         |
| <code>numeros[2]</code>   | 30           | O índice 2 corresponde ao terceiro elemento.         |
| <code>numeros[-1]</code>  | 50           | Índices negativos começam pelo final. -1 é o último. |
| <code>numeros[-2]</code>  | 40           | -2 é o penúltimo elemento.                           |
| <code>numeros[1:4]</code> | [20, 30, 40] | Começa no índice 1 (incluído) e para antes do 4.     |
| <code>numeros[0:3]</code> | [10, 20, 30] | Retorna os elementos dos índices 0, 1 e 2.           |
| <code>numeros[2:]</code>  | [30, 40, 50] | Início em 2 e fim omitido: vai até o último.         |

| Comando                         | Resultado                         | Explicação                                         |
|---------------------------------|-----------------------------------|----------------------------------------------------|
| <code>numeros[:3]</code>        | <code>[10, 20, 30]</code>         | Início omitido: do primeiro até antes do índice 3. |
| <code>numeros[:]</code>         | <code>[10, 20, 30, 40, 50]</code> | Tudo: cópia da lista inteira.                      |
| <code>numeros[-3:]</code>       | <code>[30, 40, 50]</code>         | Os três últimos elementos.                         |
| <code>len(numeros)</code>       | <code>5</code>                    | Quantidade de elementos da lista.                  |
| <code>20 in numeros</code>      | <code>True</code>                 | Verifica se o valor 20 está na lista.              |
| <code>100 in numeros</code>     | <code>False</code>                | O valor 100 não está na lista.                     |
| <code>numeros.count(10)</code>  | <code>1</code>                    | Quantas vezes o valor aparece na lista.            |
| <code>numeros.index(40)</code>  | <code>3</code>                    | Índice da primeira ocorrência do valor.            |
| <code>numeros + [60, 70]</code> | <code>[..., 60, 70]</code>        | Concatena duas listas e retorna uma nova.          |
| <code>numeros * 2</code>        | <code>[...]repetida</code>        | Repete a sequência inteira da lista.               |
| <code>min(numeros)</code>       | <code>10</code>                   | Retorna o menor valor da lista.                    |
| <code>max(numeros)</code>       | <code>50</code>                   | Retorna o maior valor da lista.                    |
| <code>sum(numeros)</code>       | <code>150</code>                  | Soma todos os valores (se forem numéricos).        |

## Métodos mais utilizados

Exemplos considerando `lista = [1, 2, 3]`.

| Método                    | Exemplo                           | Resultado                    | Explicação                                  |
|---------------------------|-----------------------------------|------------------------------|---------------------------------------------|
| <code>append(x)</code>    | <code>lista.append(4)</code>      | <code>[1, 2, 3, 4]</code>    | Adiciona um elemento ao final.              |
| <code>insert(i, x)</code> | <code>lista.insert(1, 99)</code>  | <code>[1, 99, 2, 3]</code>   | Insere x na posição i.                      |
| <code>extend(it)</code>   | <code>lista.extend([4, 5])</code> | <code>[1, 2, 3, 4, 5]</code> | Estende com os itens do iterável.           |
| <code>remove(x)</code>    | <code>lista.remove(2)</code>      | <code>[1, 3]</code>          | Remove a primeira ocorrência de x.          |
| <code>pop([i])</code>     | <code>lista.pop()</code>          | <code>3</code>               | Remove e retorna o último (ou o índice i).  |
| <code>clear()</code>      | <code>lista.clear()</code>        | <code>[]</code>              | Remove todos os elementos.                  |
| <code>index(x)</code>     | <code>lista.index(1)</code>       | <code>0</code>               | Índice da primeira ocorrência de x.         |
| <code>count(x)</code>     | <code>lista.count(2)</code>       | <code>1</code>               | Conta quantas vezes x aparece.              |
| <code>sort()</code>       | <code>lista.sort()</code>         | <code>[1, 2, 3]</code>       | Ordena em ordem crescente (altera a lista). |
| <code>reverse()</code>    | <code>lista.reverse()</code>      | <code>[3, 2, 1]</code>       | Inverte a ordem dos elementos.              |

## Ordenação: `sort()` x `sorted()`

As duas formas ordenam valores, mas se comportam de maneira diferente. Escolha **`sort()`** quando quiser alterar a própria lista; use **`sorted()`** quando precisar manter a original intacta.

## Comparação rápida

| Aspecto                  | sort()               | sorted()              |
|--------------------------|----------------------|-----------------------|
| Altera a lista original? | Sim                  | Não                   |
| Retorno                  | None                 | Nova lista            |
| Onde usar                | Método: lista.sort() | Função: sorted(lista) |
| Lista original depois    | Fica ordenada        | Permanece igual       |

Exemplo com sort() — altera a lista original:

### sort()

```
numeros = [3, 1, 4, 1, 5]
numeros.sort()
print(numeros) # [1, 1, 3, 4, 5]
```

Exemplo com sorted() — cria uma nova lista ordenada:

### sorted()

```
original = [3, 1, 4, 1, 5]
ordenada = sorted(original)
print(ordenada) # [1, 1, 3, 4, 5]
print(original) # [3, 1, 4, 1, 5] (não mudou)
```

## Função set() — remover duplicatas

A função **set()** converte uma lista em conjunto e remove valores repetidos. Conjuntos não garantem ordem; quando precisar de lista novamente, use **list(set(lista))**.

### set()

```
presencas = ['P', 'F', 'P', 'X', 'F', 'P']
unicos = set(presencas)
print(unicos) # {'P', 'F', 'X'} (sem repetição)
lista_unica = list(set(presencas))
print(lista_unica) # ordem pode variar
```

### Dica

Use set() para contar itens distintos ou limpar repetições antes de analisar dados.

## Fatiamento (slice) visual

Considere **lista = [10, 20, 30, 40, 50]** e seus índices:

| Índice | 0  | 1  | 2  | 3  | 4  |
|--------|----|----|----|----|----|
| Valor  | 10 | 20 | 30 | 40 | 50 |

| Expressão               | Resultado                 | Leitura                                |
|-------------------------|---------------------------|----------------------------------------|
| <code>lista[1:4]</code> | <code>[20, 30, 40]</code> | Do índice 1 (incluído) até antes do 4. |
| <code>lista[2:]</code>  | <code>[30, 40, 50]</code> | Do índice 2 até o final.               |
| <code>lista[:3]</code>  | <code>[10, 20, 30]</code> | Do início até antes do índice 3.       |
| <code>lista[-3:]</code> | <code>[30, 40, 50]</code> | Os três últimos elementos.             |

### Regra de ouro do fatiamento

O índice inicial é INCLUÍDO; o índice final NÃO é incluído.

### Exercício: carrinho de compras

Sistema de carrinho que permite adicionar, remover, atualizar a quantidade, exibir o total e sair. Cada item é uma lista no formato **[nome, preço, quantidade]** — ex.: `['Arroz', 4.99, 2]`.

**carrinho.py**

```
carrinho = [] # cada item: [nome, preco, quantidade]

def adicionar():
    nome = input('Produto: ')
    preco = float(input('Preço: '))
    qtd = int(input('Quantidade: '))
    carrinho.append([nome, preco, qtd])
    print('Produto adicionado!')

def remover():
    nome = input('Produto a remover: ')
    for item in carrinho:
        if item[0] == nome:
            carrinho.remove(item)
            print('Removido!')
            return
    print('Produto não encontrado.')

def atualizar():
    nome = input('Produto: ')
    for item in carrinho:
        if item[0] == nome:
            item[2] = int(input('Nova quantidade: '))
            print('Quantidade atualizada!')

def exibir():
    total = 0
    for nome, preco, qtd in carrinho:
        subtotal = preco * qtd
        total += subtotal
        print(nome, qtd, 'x', preco, '=', subtotal)
    print('TOTAL:', total)

while True:
    print('1-Adicionar 2-Remover 3-Atualizar 4-Exibir 5-Sair')
    opcao = input('Escolha: ')
    if opcao == '1': adicionar()
    elif opcao == '2': remover()
    elif opcao == '3': atualizar()
    elif opcao == '4': exibir()
    elif opcao == '5': break
```

**Dica final**

Listas são extremamente versáteis e aparecem em quase todo programa. Domine os métodos e o fatiamento para manipular dados com eficiência.

**Prof. Gustavo Franz (Science/Biology)***Python Developer in Progress**Building Educational Solutions with Python*

Professor de Ciencias e Biologia desde 2013 — atualmente estudando Programacao.

Eu crio estes resumos para organizar os assuntos e estudar de forma clara e objetiva. Estou compartilhando porque eles tambem podem ajudar outros desenvolvedores que estao comecando. Bons estudos — vamos aprender juntos!

**GitHub:** [github.com/GustaFranz](https://github.com/GustaFranz)

**Exercicios Python:** [github.com/GustaFranz/python\\_exercises](https://github.com/GustaFranz/python_exercises)

Para quem quiser ver a resolucao dos meus exercicios em Python.

**EasyAnsi:** [github.com/GustaFranz/easyansi](https://github.com/GustaFranz/easyansi)

Para quem quiser codificar personalizando com cores e estilos de forma mais facil, pratica e intuitiva.